

Executive Business Presentation

Ram Sundar



Agenda

- Business problem and operational risks
- System architecture and data flow
- Key insights from SQL and EDA
- Model performance in business terms
- Financial impact and ROI estimation
- Deployment and scaling strategy
- Risk management and compliance plan

Business problem and operational risks

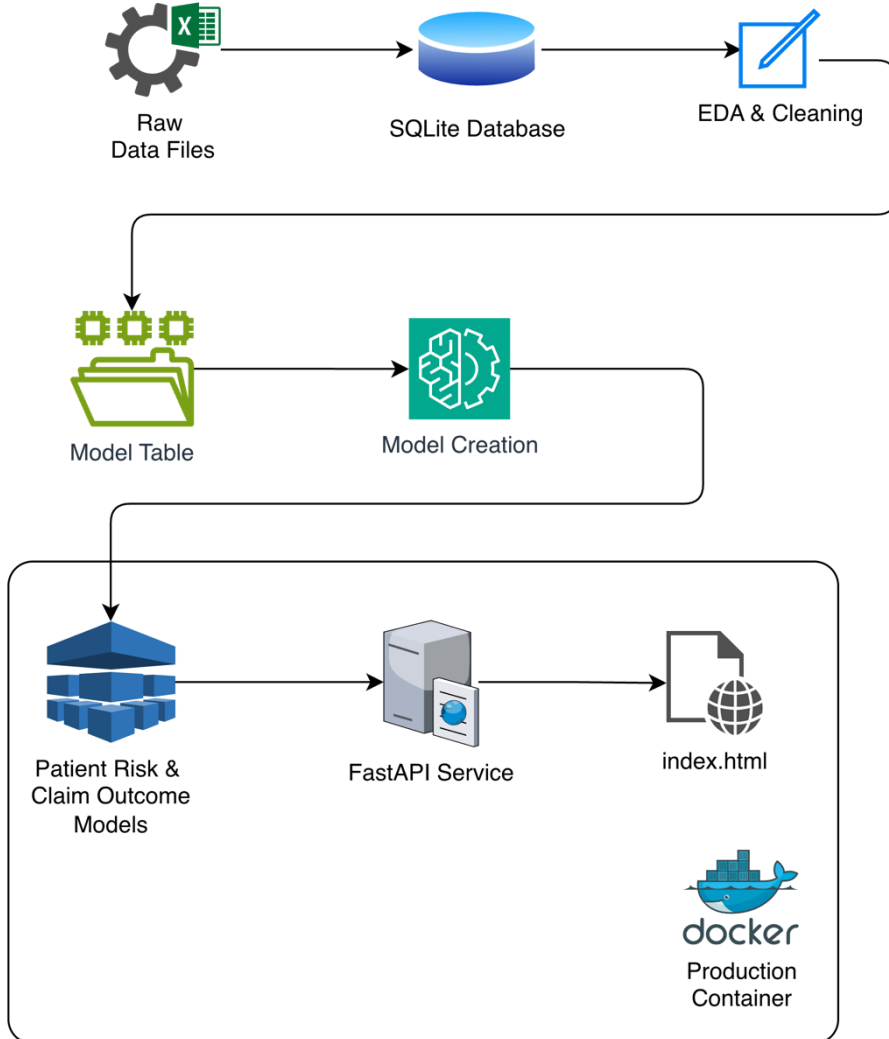
The Problem

- Healthcare organizations face two interconnected operational challenges:
- **Patient Risk Triage:** Hospitals cannot efficiently prioritize patients at check-in. Without automated risk stratification, clinical staff rely on subjective assessment, leading to delayed care for high-risk patients and misallocated resources across departments.
- **Insurance Claim Outcomes:** Claims processing is reactive, not predictive. Rejected or pending claims (claim statuses: Paid / Pending / Rejected) create revenue leakage, delayed cash flow, and administrative overhead. Billing teams discover rejections only after submission.

Operational Risks Identified

- **Department-level risk concentration:** Certain departments (Cardiology, ICU, ER) handle disproportionately high volumes of "High" risk visits, requiring targeted staffing
- **Insurance provider variability:** The top 5 providers show significantly different rejection rates, indicating that claim preparation must be provider-aware
- **Payment delays:** Average payment days vary by insurance provider, impacting cash flow predictability
- **Revenue realization gaps:** Revenue realization ratios (approved/billed) differ significantly by department; some departments show high billing with zero approved amounts
- **Data integrity issues:** Orphaned visits (no billing record), missing payment days, and invalid length-of-stay values detected in source systems

System architecture and data flow



- Raw CSV data ingested into SQLite database with indexed foreign keys
- 13 engineered features added in Phase 2: `visit_frequency`, `avg_length_of_stay`, `days_since_registration`, `total_billed`, `avg_payment_days`, `provider_rejection_rate`, `doctor_rejection_rate`, `visit_month`, `visit_day_of_week`, `visit_quarter`, `is_weekend`, `days_between_reg_and_visit`, `billing_delay_days`
- Missing values imputed with hierarchical median strategy (patient → department → global)
- Time-based 80/20 train/test split (no shuffling — prevents data leakage)
- Models serialized as .pkl pipelines with corresponding `feature_schema.json` for validation
- Every API prediction logged with UUID, model version, timestamp, SHA-256 feature hash, label, and confidence score

Key insights from SQL and EDA (1/2)

Analysis	Key Finding
Department visit volume	Top departments by volume identified; enables capacity planning
Length of stay	Significant variation across departments — ICU and specialty wards have highest average stay
High-risk concentration	Certain departments carry a disproportionate share of High-risk visits
Doctor workload	Top 10 doctors handling the most High-risk visits identified — workforce balancing opportunity
City-level utilization	Average visits per patient varies by city — geographic resource allocation insight
Insurance billing	Total billed amounts differ significantly by provider
Claim rejection rates	Rejection rates vary across insurance providers — some providers are harder to collect from
Payment delays	Average payment days differ by insurer — cash flow planning signal
Revenue realization	Department-level approved/billed ratios reveal where money is being left on the table
High-bill zero-approval	Flagged visits with billed amount > \$1,000 but \$0 approved — potential billing errors

Key insights from SQL and EDA (2/2)

Finding	Detail
Target distributions	risk_score: ~50% Low / ~30% Medium / ~20% High; claim_status: ~65% Paid / ~15% Pending / ~20% Rejected
Class imbalance	Both targets are imbalanced; addressed via SMOTE (training only) and PyCaret's fix_imbalance
Missing values	approved_amount, payment_days, length_of_stay_hours had missing values — imputed with cascading median strategy
Outliers	Detected in billed_amount, payment_days, length_of_stay_hours via IQR method
Feature correlations	All correlations with risk_score are weak ($\max r \approx 0.025$) — expected for categorical target; tree models handle this
Billing date anomaly	billing_date is largely frozen (same date) — billing_delay_days flagged as unreliable and excluded

Model performance in business terms

Model 1

Model 1: Patient Risk Classification

Purpose: Predict visit risk (Low / Medium / High) at the moment of patient check-in

Attribute	Detail
Algorithm	XGBoost (gbtree) via PyCaret pipeline
Features	chronic_flag, gender, visit_frequency (3 features — all available at check-in)
Best Macro F1	0.3246 (best among all model variants tested)
Why this model?	Highest Macro F1 ensures High and Medium risk patients are identified, not just the majority Low class
Training approach	Time-based split, SMOTE for class balancing, PyCaret-optimized hyperparameters
Key hyperparameters	learning_rate=0.15, max_depth=8, n_estimators=10, subsample=0.5, reg_alpha=2

Models benchmarked: Logistic Regression (baseline), XGBoost (default), Random Forest, XGBoost (GridSearchCV tuned), all with and without SMOTE, demographic-only experiments — PyCaret XGBoost selected as production model.

Business translation:

- Enables **automated triage prioritization** — High-risk patients flagged immediately at check-in
- Lean 3-feature design means **zero additional data collection** needed at point of care
- SHAP explainability allows clinical staff to understand *why* a patient is flagged

Model performance in business terms

Model 2

Model 2: Claim Outcome Classification

Purpose: Predict claim status (Paid / Pending / Rejected) before submission

Attribute	Detail
Algorithm	PyCaret auto-selected best classifier (tuned on MCC)
Features	12 inputs: age, gender, city, insurance_provider, chronic_flag, department, visit_type, length_of_stay_hours, billed_amount, payment_days, visit_frequency, lag_time
Optimization metric	Matthews Correlation Coefficient (MCC) — penalizes majority-class bias
Training approach	Time-based split, SMOTE for Pending/Rejected oversampling, PyCaret feature selection (top 80%), multicollinearity removal

Business translation:

- Pre-submission claim screening — flag likely Rejected claims for review before filing
- Identify which provider/department/amount combinations trigger rejections
- Reduce rejection-related rework and appeals overhead

Financial impact and ROI estimation

Revenue Leakage Quantified from the dataset (25,000 billing records):

- Rejected claims (~20% of claims) represent direct revenue loss
- Pending claims (~15%) create cash flow delays and administrative overhead
- High-bill-zero-approval visits flagged as potential billing errors

Impact Area	Mechanism	Estimated Benefit
Claim rejection reduction	Pre-screen claims before submission; correct documentation, coding, or submission timing for likely-rejected claims	Even a 10-20% reduction in rejections directly recovers revenue
Faster payment cycles	Predict Pending claims → proactively follow up with insurers	Reduced days-sales-outstanding (DSO); improved cash flow
Triage efficiency	Auto-classify risk at check-in → prioritize High-risk patients	Reduced adverse events, shorter ED wait times, better resource utilization
Staff productivity	Automated risk scoring replaces manual chart review	Nurse/admin time redirected to patient care
Compliance cost avoidance	Audit trail, SHAP explainability, validated inputs	Reduces regulatory risk and audit preparation effort

Cost of Implementation

- Infrastructure: Single Docker container, 1 worker process — minimal compute cost — auto-scale as required by visit patterns of patients
- Integration: REST API with standard JSON — integrates with any EMR/billing system
- Maintenance: Periodic retraining when drift detected (monthly/quarterly cadence)

Deployment and scaling strategy

Current Deployment

Component	Detail
Framework	FastAPI + Uvicorn
Containerization	Docker (Python 3.11-slim, libgomp for XGBoost)
Endpoints	2 prediction endpoints, 3 health checks, 1 metadata endpoint, 1 web UI
Input validation	Schema-based validation (validate.py) — catches missing fields, range violations, unseen categories; returns all errors at once
Logging	Every prediction logged with UUID, model version, UTC timestamp, SHA-256 feature hash, prediction label, confidence score
Privacy	No raw patient data in logs — only SHA-256 hashes; patient IDs are never accepted as API inputs
UI	Built-in HTML/Bootstrap testing interface with dynamic form (loads valid categories from /metadata/claim-outcome)

Scaling Strategy

Phase	Approach
Current (v1.0)	Single container, 1 worker — keeps RAM predictable (PyCaret models are large)
Horizontal scale	Add container replicas behind a load balancer (not more workers per container)
Production readiness	Health endpoints (/health, /health/patient-risk, /health/claim-outcome) enable orchestrator liveness/readiness probes
Model updates	Swap .pkl file → update version string in Api.py → restart container (zero-code-change deployment)
Schema evolution	feature_schema.json versioned alongside models; validation layer auto-enforces

Risk management and compliance plan (1/3)

Model Risk Management

Risk	Mitigation
Data drift	Evidently AI drift detection on all model features + target distribution (DataDriftPreset + TargetDriftPreset); HTML drift reports generated
Concept drift	Time-based train/test split validates on future-like data; retraining triggers defined
Class imbalance shift	SMOTE applied at training only; production prediction distribution monitored via audit log
Feature schema violations	Real-time validation layer rejects invalid inputs with HTTP 422; all errors surfaced at once
Model staleness	Defined retraining triggers: >30% features drifting, target distribution shift, F1 drop >10%, schema mismatch rate >1% in 24h

Retraining Protocol

Documented checklist: collect new labeled data → re-run feature engineering → retrain → evaluate against baseline → update schema → increment version → swap pkl → restart → smoke test → archive old model

Risk management and compliance plan (2/3)

Compliance & Governance

Area	Implementation
Audit trail	Every prediction logged: UUID, model+version, timestamp, SHA-256 hash, label, confidence
Data minimization	No PII in logs; SHA-256 hashes enable traceability without storing patient data
Log retention	Recommended: 0–90 days hot, 91–365 days cold, >365 days delete/anonymize
Model cards	Full documentation for both models: training data, evaluation metrics, limitations, ethical considerations
Explainability	SHAP waterfall plots for individual predictions; feature importance rankings documented
Fairness monitoring	Gender bias check (risk model uses gender as input), geographic bias check (city in claim model), age-bracket analysis recommended
Prohibited uses	Models must NOT be sole basis for treatment decisions, automatic claim rejection without human review, or denial of care/coverage
Intended uses	Clinical triage support, claims workflow automation, operational resource planning

Risk management and compliance plan (3/3)

Known Limitations

Limitation	Impact
Risk model uses only 3 features	Coarse risk proxy; does not capture diagnosis severity or comorbidity count
Binary chronic_flag	Patients with 1 vs. 3 chronic conditions treated identically
payment_days may not be available at prediction time	Requires imputation strategy at inference
Negative lag_time in training data	Data quality issue; schema enforcement planned for v1.1
25,000 rows is modest for healthcare AI	Minority classes (High-risk: ~5K, Rejected: ~3.8K) may overfit
PyCaret auto-selection non-deterministic	Model type pinning recommended for v2.0
No real-time retraining or online learning	Static model files; requires explicit retrain-redeploy cycle

Verified Governance Artifacts

All artifacts confirmed present: both .pkl model files, both feature schema JSONs, validate.py, Api.py, predictions.log, both HTML drift reports.

